LLMProjectMemory Development Handbook

Architecture usage troubleshooting and continuing development

OchsSoft | 26 October 2026

This handbook explains the project from everyday use to the code and data that implement it. Use it to set up a development checkout, follow a request or document through the system, locate the right extension point, make a change, and verify the result.

Begin with the project overview and architecture. The later chapters provide exact local commands, user journeys, worked development examples, diagnostics, recovery steps, known gaps, and a pinned source reference. Technical terms are explained where they are used.

The documented source is OchsSoft/LLMProjectMemory at c0392ff94c15792c9d693df10f02c9b2ff2d4d29. Source-defined behavior, observed evidence, planned capabilities, and remaining platform work are distinguished. The quick operations guide remains a separate short reference.

Contents

Understand the project

Understand the project....3
Architecture and information contracts....4

Use and develop the project

Local development and review....7
Using the reference in practice....8
Developing the design into a reliable implementation....9

Diagnose improve and verify

Gaps diagnosis and recovery....11
Source reference....12

Understand the project

What this repository delivers

LLMProjectMemory is a design reference and a reusable skill specification for portable, linked project knowledge. It is not a running memory server, database package, command line program or search engine. The committed implementation consists of Markdown documents and a skill directory. There is no dependency manifest, executable retrieval compiler, parser, database migration or automated test suite in this baseline.

Think of its proposed memory graph as a box of small index cards. Each card says one useful thing, names the evidence behind it, says whether it is known or inferred, and points to related cards. A retrieval pack is a deliberately small selection of those cards for a particular task. This is intended to avoid repeatedly reading an entire project or mistaking an old conversation for present authority.

The important distinction is between a pattern and a product. The schema in `references/note_schema.md` describes what an implementation should accept. The retrieval document describes the format a compiler should produce. Neither document proves that a compiler exists. An operator can study or adapt this material, but cannot start a background service from this repository.

Two uses with different ownership

Build-time memory helps a developer or coding assistant understand architecture, constraints, domain rules, migrations and known problems. The application being developed need not use AI. Runtime memory belongs to an application whose user-facing behavior uses an LLM. Runtime memory needs durable application storage, explicit scopes and validated state changes; the skill specifically recommends that Markdown should not be the runtime source of truth.

A hybrid project might use both, but the two must not become one unfiltered transcript archive. A developer preference is not automatically a customer preference. A generated story is not automatically an accepted domain fact. A confidence label is a statement about evidence, not a permission to act.

Workspace boundary

The workspace currently uses a canonical durable project memory system and forbids ad hoc operational memory files. The skill's example instruction to create `docs/llm-memory/` is therefore a reusable proposal, not authorization to create parallel operational vaults in existing workspace projects. Follow the receiving project's governing instructions first. This handbook adds ordinary project documentation; it does not adopt a new canonical store.

The project has no supplied canonical project key in the workspace list. Do not infer a key from the repository name. Any future integration requiring an exact identity must first resolve that identity explicitly.

Read in this order

Read `README.md` for scope, `docs/project-brief.md` for the intended pipeline, then the skill and its four references for contracts. `docs/example-project-prompts.md` preserves initial rollout prompts. Treat those prompts as historical examples and proposed work, not evidence that their target projects now implement every requirement. This chapter and the architecture chapter explain how to review those requirements independently.

Architecture and information contracts

The proposed pipeline

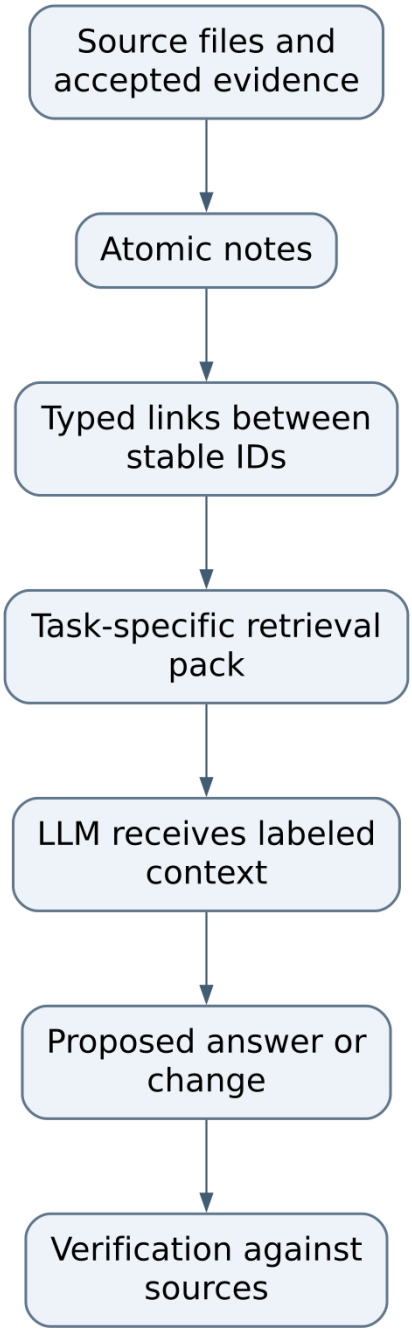


Figure 1 The proposed pipeline

This is a design flow, not an executable deployment diagram. No process, port, queue or network endpoint in it is implemented here. Its reliability depends on preserving the difference between evidence, interpretation and action.

Note schema

The required frontmatter fields are `id`, `type`, `status`, `confidence`, `source`, `links`, and `last_verified`. Frontmatter is the metadata block between opening and closing `---` lines at the beginning of a Markdown document. The note body supplies the actual claim and enough local context to understand it without opening every linked note.

Field	Contract and maintenance implication
`id`	Stable identity. Rename the display title without casually changing this ID.
`type`	One of decision, constraint, gotcha, workflow, domain-fact, open-question or runtime-memory. This classifies the claim's purpose.
`status`	confirmed, inferred, disputed, deprecated or open-question. This controls how retrieval should present it.
`confidence`	high, medium or low. High confidence is not a substitute for provenance or current verification.
`source`	A list of source pointers. A parser should verify these are well formed and meaningful in the receiving project.
`links`	Wiki-style links using stable IDs. The examples use `[[related-note-id]]`.
`last_verified`	A date. It indicates when evidence was checked, not an automatic expiration policy.

There is a useful unresolved detail: the README's YAML example writes unquoted wiki links. YAML parsers may interpret brackets as collection syntax rather than strings. Before building a parser, choose a strict serialization and fixtures, for example quoted link strings. Document that choice rather than assuming every frontmatter parser accepts the illustrative syntax identically.

Retrieval behavior

The skill requests directly matched notes, one-hop expansion of links, provenance and status metadata, deprecated-note exclusion by default, and warnings for disputed information. A one-hop expansion means follow a direct link once; it does not mean recursively load the entire graph. The references additionally say to keep the pack small and prefer confirmed, high-confidence notes.

The output reference separates Canon Facts, Inferred Notes, Disputed Notes and Open Questions. The runtime reference also names Recent Events. These are compatible purposes but not a complete executable selection algorithm. There are no specified scoring weights, byte/token budgets, tie breakers, permission filters or link-cycle algorithms. Those are real design gaps, not hidden defaults.

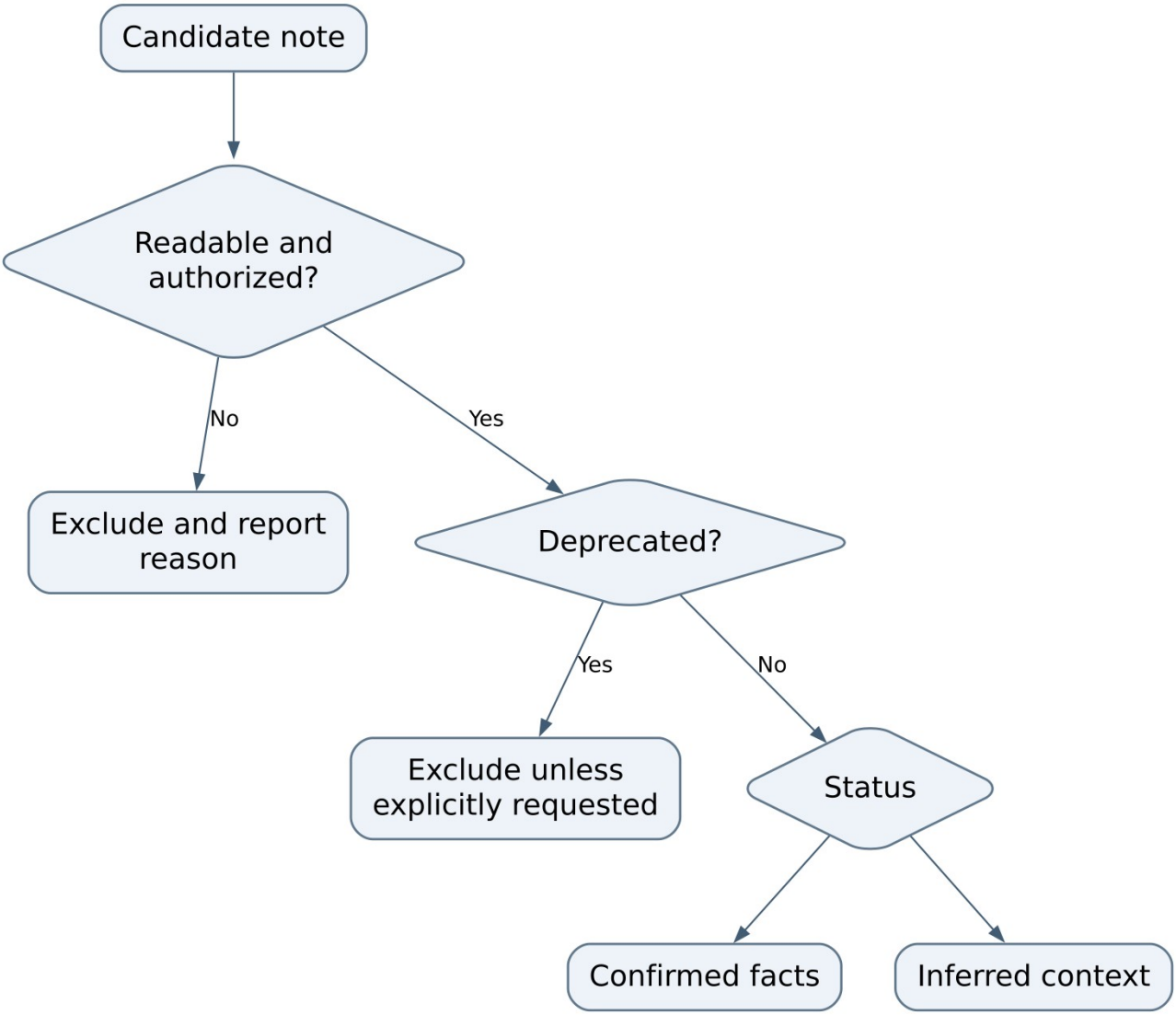


Figure 2 part 1 of 2 Retrieval behavior

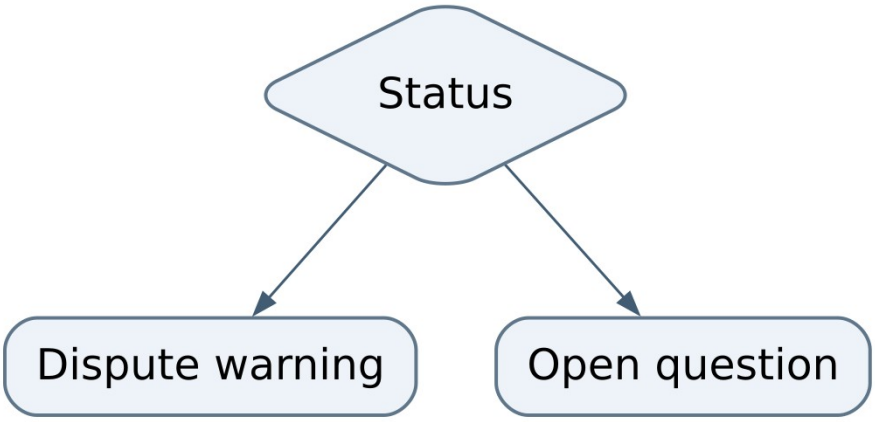


Figure 2 part 2 of 2 Retrieval behavior

The permission check in this diagram is a required design consideration for a future implementation, not an implemented feature of this repository. The current documents do not define an authorization engine.

Runtime storage proposal

`runtime_memory.md` proposes `session_events`, `memory_notes`, `memory_links`, and optional embeddings. Events retain evidence; notes distill it; links relate notes; embeddings help find candidates. Embeddings must remain a derived retrieval aid, since a similarity result cannot establish truth. Narrative generation should not directly mutate accepted canon: extraction and validation must intervene. No SQL schema, API or database implementation for these proposed names is committed.

Local development and review

Start with a clean explicit checkout

This is a document repository. On NixOS, Git and a text editor are sufficient to review the current artifact. There is no environment to activate, no package installation command and no native service to launch. Inspect `git status --short --branch`, the latest commit and the file inventory before changing anything. Use a topic branch and preserve unrelated changes. Publication targets the canonical primary branch through a human-reviewed PR; routine hosted validation is not a substitute for local review.

Do not run application setup commands copied from an example prompt in an unrelated working directory. A proposed vault is not a supplied dataset, and a suggested integration is not an installed adapter. The ordinary editing loop is to read the relevant contract, edit the smallest related document, check internal references, and review the diff for unsupported claims and private material.

Reproduce the content review

These read-only checks work in Bash from the repository root:

```
git status --short --branch
git ls-files
rg -n '^(#|name:|description:)|session_events|memory_notes|memory_links' README.md docs skills
git diff --check
```

The search locates contracts rather than validating YAML. `git diff --check` finds whitespace errors, not incorrect architecture. There is no committed linter or test command. A future schema validator would require a deliberate dependency choice and tests; installing an arbitrary YAML parser does not make it an established project toolchain.

What counts as a meaningful review

Check that each field's meaning is consistent in the README, skill and schema reference. Confirm that a status cannot accidentally become confidence, that runtime proposals are labeled proposals, and that examples contain no secrets, real customer data or private machine paths. Inspect wiki-link syntax with any parser selected by a future implementation. Follow every local documentation link from the receiving document's directory, because relative paths are resolved there.

Do not claim NixOS runtime support for an application that does not exist. The current documentation can be read and edited on NixOS. An eventual executable implementation would need a reproducible

development environment, supported parser versions and native checks before claiming that larger support boundary.

Using the reference in practice

A source backed decision example

Suppose a receiving project's approved architecture says that one durable store owns business records and a search index can be rebuilt. The useful memory is one concise decision describing ownership and its consequence, with a stable ID and a pointer to the accepted architecture or code. It is not an entire chat transcript. If the evidence only shows the search index configuration, the ownership conclusion may be inferred rather than confirmed.

Start by reading the receiving project's instructions and current memory convention. If it already has a canonical system, record or retrieve the decision there using its verified identity and allowed workflow. Study this repository's status, provenance and retrieval ideas without creating a competing vault. If a separate future project explicitly adopts Markdown memory, its maintainers must first define the permitted location, schema parser and review process.

Construct a pack by hand for understanding

Choose a concrete task such as changing a save format. Find the relevant schema decision, migration constraint and known compatibility issue. Follow only directly useful links. Place confirmed evidence under Canon Facts, hypotheses under Inferred Notes, conflicting records under Disputed Notes, and unresolved compatibility questions under Open Questions. Preserve source pointers in every section. This manual exercise demonstrates the proposed contract without pretending an automated retriever has run.

A small pack should still carry enough context to avoid dangerous ambiguity. “Use the new schema” is unusable without a version and scope. “The version 2 importer accepts version 1 fixtures; tested in this named test at this revision” is reviewable. Conversely, packing every note into context defeats the design's aim and makes conflicting claims harder to see.

A runtime example still a design

An application receives a user preference. It first records the event in its own durable store, then proposes a scoped note connected to that event. Validation decides whether the wording and scope are justified. Retrieval can later include the note with its confidence and evidence. If the user corrects the preference, the system should preserve provenance while preventing the older note from silently winning. The repository recommends these separations but does not define correction transactions, deletion behavior or privacy enforcement.

Finishing a session

There is no process to stop. Save document edits, review `git diff`, and leave a clear local validation result. Do not commit a generated operational vault or raw conversations as an incidental product of reading the skill. If a future implementation produces packs, treat them as derived data unless an explicit audit contract says otherwise; do not accidentally make a stale pack the new source of truth.

Developing the design into a reliable implementation

Recipe resolve the link representation

Begin with `note_schema.md`, the README example and the skill's example. Choose whether links are strings containing wiki syntax or a structured list of IDs. Add fixtures showing a valid note, a quoted link, missing metadata, an unknown status, duplicate IDs and a dangling link. Define whether invalid notes fail a build or are omitted with a visible diagnostic. Then update every example to the same representation. Acceptance is reproducible parsing and clear rejection, not merely that a Markdown renderer displays the text.

This is a proposed implementation recipe. The current repository has no parser test harness, so adding one is real new product work and needs an explicit language/toolchain decision. Prefer a small portable validator before designing an entire database or user interface.

Recipe define deterministic retrieval

Specify matching inputs, maximum output size, stable ordering, permission boundaries, status routing and one-hop expansion before writing code. Create fixtures with a link cycle, two equal matches, an inferred note linked to a confirmed note, a deprecated note and a disputed amendment. The expected pack should be byte-for-byte stable for the same inputs. A linked confirmed note must not promote its inferred neighbor to canon. A missing source should produce an explicit problem rather than disappear unnoticed.

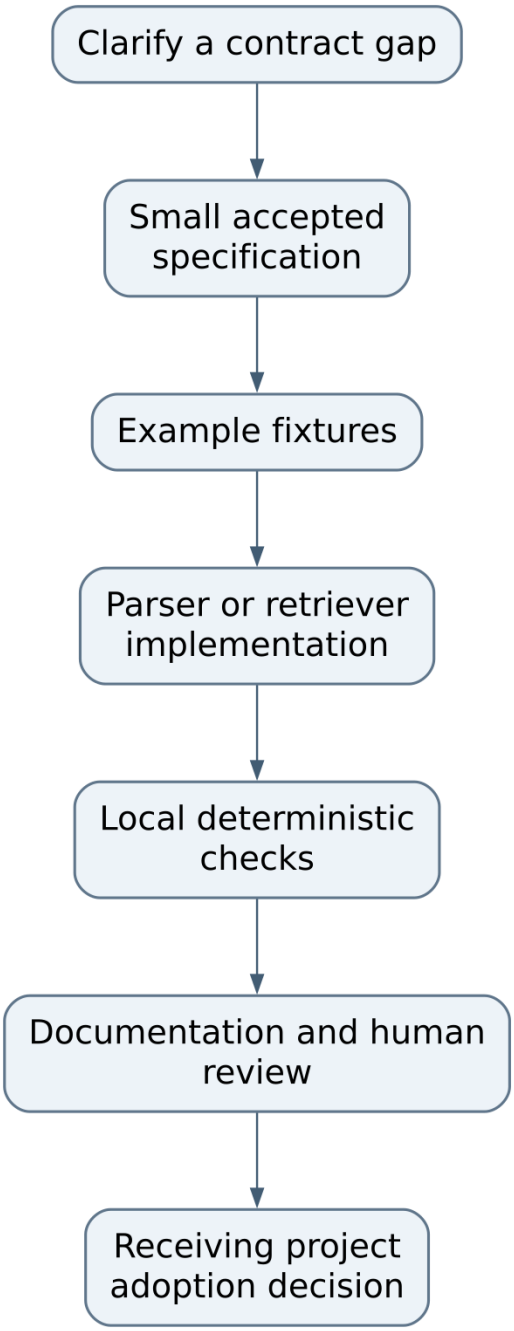


Figure 3 Recipe: define deterministic retrieval

Recipe define corrections before runtime integration

A runtime implementation must specify who can confirm a note, how an event is linked, whether old facts are superseded or deleted, and how a failed update is recovered. Write scenarios for “user corrected a fact,” “source became unavailable,” “two authorized sources disagree,” and “retrieval index was rebuilt.” Storage and retrieval must honor the same scope. Test canon changes separately from narrative generation.

Keep raw evidence access distinct from the compact pack. A source pointer should permit an authorized reviewer to investigate a claim, but should not automatically expose private content to

every consumer. The present pattern needs an explicit privacy/access model before it is suitable as a multi-user runtime product.

Maintaining the skill

The skill is source material in this repository, not a secretly installed agent setting. Changes to its instructions should be reviewed as behavior changes: a new default vault location or automatic mutation rule can affect every adopter. Keep the description narrow enough to distinguish documentation design from actual memory-server operation. Update references together, preserve historical evidence where useful, and state migration implications for any future schema change.

Gaps diagnosis and recovery

Current limitations and next steps

Priority	Gap and affected material	Observable acceptance
First	Illustrative frontmatter may parse links inconsistently; README, skill, note schema	One specified representation, valid/invalid fixtures and deterministic parser behavior
First	No executable validator or retrieval compiler	Explicit toolchain, local checks and documented invocation; no undocumented background service
First	No access-control or correction protocol for runtime use	Scoped scenarios demonstrate permitted reads and validated corrections without canon contamination
Next	Retrieval budget/order/conflict rules are qualitative	Stable fixture packs cover truncation, ties, cycles, disputes and deprecated records
Next	No schema versions or migration contract	Old fixtures migrate or fail clearly with retained original evidence
Later	No embedding/index implementation	Index rebuild proves original records and provenance remain canonical

Troubleshooting without inventing a runtime

Symptom	Evidence to inspect	Safe response
There is no start command	Tracked file list and absence of package manifest	Use the design documents; do not invent a server invocation

Symptom	Evidence to inspect	Safe response
A parser rejects an example	Exact YAML bytes and chosen parser version	Preserve original example, isolate a minimal fixture and standardize syntax
A pack says an inference is a fact	Note status and pack section	Correct section routing and test the case before relying on the pack
Old information keeps returning	Status, source revision and index policy	Resolve supersession explicitly; rebuild only derived retrieval data
A rollout prompt conflicts with project rules	Receiving repository instructions and current authority	Follow the receiving project's rules; amend the reusable example if misleading
Links lead nowhere	Stable IDs versus filenames and relative paths	Repair references deliberately; do not create duplicate identities casually

Preservation and recovery

The committed Markdown and Git history are the repository's authoritative artifacts. Back up the repository with its history and any explicitly adopted external evidence separately. A document export is useful for reading but cannot replace editable contracts and source links. There are no database volumes, user accounts or running sessions to back up here.

Before undoing a document change, compare the exact diff and preserve unrelated work. Recover a specific known revision into a separate file or clean checkout for comparison instead of resetting an active working tree. If a future application adopts this pattern, its event store needs its own tested backup and restore procedure; copying this reference repository cannot restore that application's memory.

A successful recovery means the relevant source can be read, metadata meaning is consistent, links resolve, and no unsupported claim has been elevated to accepted truth. It does not mean an unimplemented runtime has become operational.

Source reference

This handbook describes committed source in OchsSoft/LLMProjectMemory at `c0392ff94c15792c9d693df10f02c9b2ff2d4d29`. Paths below are relative to this repository. Source-defined behavior is distinguished from a fresh runtime verification; no application or game was launched to author these chapters. The existing quick operating guide remains available separately.

Reading the evidence

A source file explains what the program is designed to do. A test states an expectation and may exercise only one layer. A README describes an intended workflow and can lag implementation. When these disagree, inspect the exact revision and the executable code, reproduce the disagreement in a safe local test, and correct the relevant documentation with the change. Do not treat a passing unit test as proof of rendering, sound, external connectivity, deployment, or save recovery.

File and symbol index

File	Responsibility	Entry points and locations
README.md	Scope and illustrative note model	L1 to L46
docs/project-brief.md	Design goals and proposed pipeline	L1 to L81
docs/example-project-prompts.md	Historical rollout examples	L1 to L19
skills/markdown-memory-graph/SKILL.md	Reusable skill specification	L1 to L73
skills/markdown-memory-graph/references/note_schema.md	Frontmatter fields and meaning	L1 to L23
skills/markdown-memory-graph/references/retrieval_pack_format.md	Output sections and selection guidance	L1 to L26
skills/markdown-memory-graph/references/build_time_memory.md	Development knowledge scope	L1 to L16
skills/markdown-memory-graph/references/runtime_memory.md	Proposed runtime records and canon validation	L1 to L19

Keeping the handbook useful

When code changes, update the chapter that explains its contract, the relevant diagram, the change recipe, and this pinned source register. Keep the Word and PDF editions synchronized with the Markdown chapters. Record what was actually tested, what remained unverified, and which saved data must survive an upgrade. A future maintainer should be able to trace an instruction to a file or a test without relying on a previous conversation.